Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer e-ISSN: 2548-964X
Vol. 4, No. 9, September 2020, hlm. 2791-2800 http://j-ptiik.ub.ac.id

Pengembangan Sistem *Management* Dalam Melakukan Penugasan Berbasis Web

Bobby Ardian Ekaputra[1], Fajar Pradana[2], Djoko Pramono[3]

Program Studi Teknik Informatika, Fakultas Ilmu Komputer, Universitas Brawijaya
Email: [1]biboibob@student.ub.ac.id, [2]fajar.p@ub.ac.id, [3]djoko.jalin@ub.ac.id

Abstrak

Memahami pentingnya kontrol terhadap berbagai aspek dalam melakukan penugasan dalam menyelesaikan suatu tujuan, dibutuhkan pengelolaan yang baik agar penugasan dapat berjalan sesuai dengan yang diharapkan. Manajemen dibutuhkan sebagai prinsip untuk melakukan setiap perencanaan, terdapat beberapa faktor yang membuat manajemen harus dilakukan dalam penugasan. Penyelesaian penugasan yang beragam yang membutuhkan generalisasi sehingga pengerjaan dapat terorganisir dengan baik. Selain itu, pengontrolan dari segi waktu, prioritas, dan penugasan minor menjadi kebutuhan dalam manajemen sehingga pengerjaan penugasan dapat sesuai dengan harapan. Projek dapat digunakan sebagai penerapan dalam mengerjakan penugasan dikarenakan upaya dalam pengerjaan yang kolaboratif dan pelibatan perencanaan dalam mencapai suatu tujuan. Sistem manajemen diharapkan dapat menjadi wadah dan sarana untuk membantu pengelolaan penugasan menjadi lebih baik dari awal pengerjaan hingga akhir. Sistem akan dibangun berbasis basis *website* sehingga dapat diakses melalui perangkat keras seperti laptop atau smartphone. Hasil dari penilitan dengan menggunakan kuisioner terbagi menjadi pengguna yang belum dan sudah mengetahui sistem berbasis *team-collaboration*, dari hasil respon yang diterima dan dilakukan perhitungan menggunakan metode *System Usability Scale* (SUS) didapatkan bahwa pengguna yang belum mengetahui sistem berbasis *team-collaboration* mendapatkan rata-rata nilai 65 dan pengguna yang sudah mengetahui sistem berbasis *team-collaboration* mendapatkan rata-rata nilai 74,2.

Kata kunci: Manajemen, Penugasan, Projek, Sistem manajemen, *team-collaboration*

Abstract

Understanding the importance of control over various aspects of conducting an assignment in completing a goal, good management is needed so that the assignment can run as expected. Management is needed as a principle to do every planning, there are several factors that make management must be done in the assignment. Completion of various assignments that require generalization so that workmanship can be well organized. In addition, control in terms of time, priority, and minor assignments becomes a necessity in management so that the assignment work can be in line with expectations. Projects can be used as applications in doing assignments due to collaborative work and planning involvement in achieving a goal. The management system is expected to be a place and facility to help the management of assignments to be better from the beginning of the process until the end. The system will be built based on a website so that it can be accessed through hardware such as laptops or smartphones. The results of the research using a questionnaire are divided into users who have not and already know a team-collaboration based system, from the responses received and calculated using the System Usability Scale (SUS) method, it is found that users who do not know the team-collaboration based system get average an average score of 65 and users who already know a team-collaboration-based system get an average value of 74.2.

Keywords: *Management, Assignments, Projects, Management systems, team collaboration*

1. PENDAHULUAN

Manajemen merupakan proses merancang dan memelihara lingkungan dimana setiap

individu bekerjasama secara efisien mencapai suatu tujuan. Setiap manajemen yang dilakukan terdapat seorang penanggung jawab yang berperan sebagai pembuat keputusan untuk menggerakan setiap individu untuk memberikan kontribusi terbaik untuk tujuan kelompok. Untuk melakukan manajemen secara optimal diperlukan proyek sebagai metode manajemen bertujuan menyelesaikan permasalahan dan pengerjaan tugas dengan pertimbangan prioritas, sumber daya yang memadai, dan hasil akhir yang dapat diukur (Nigel Ostime, 2017). Projek sebagai manajemen memerlukan pertimbangan dalam setiap pengambilan keputusan sehingga harus terdapat perhitungan risiko untuk meminimalisir kesalahan selama pengerjaan proyek. Kondisi proyek yang mengarah terjadinya risiko disebabkan kesalahan manajemen, ketidakmampuan, dan ketergantungan pihak eksternal (Keshk, Maarouf, & Annany, 2018).

Terdapat metodologi yang populer digunakan dalam projek manajemen adalah *Scrum* dan *Kanban*. *Scrum* pada dasarnya merupakan metodologi pengembangan perangkat lunak yang menggunakan prinsip *agile*, kebutuhan pengembangan dengan metode *scrum* adalah untuk menangani pengerjaan bersifat *rapid-change*, pengerjaan yang dilakukan pembatasan waktu pengerjaan (*deadline*) sehingga memaksakan tugas-tugas kompleks dipecah menjadi bagian kecil, dan membantu tim dalam melakukan pembelajaran secara cepat. *Kanban* merupakan metodologi perangkat lunak yang memiliki alur kerja berkelanjutan yang membuat pengerjaan tim cekatan dan dapat beradaptasi pada perubahan prioritas. *Kanban* dasarnya memprioritaskan tugas terpenting untuk diberikan fokus untuk mengurangi risiko ketidak lengkapan. Pengerjaan dengan metode ini juga menekankan untuk melakukan pekerjaan yang tepat di waktu yang tepat sehingga dilakukan secara *continuous* untuk meningkatkan alur kerja (Lei, Ganjeizadeh, Jayachandran, & Ozcan, 2017).

Setiap organisasi, kelompok, atau perusahaan memiliki pertimbangan terkait projek manajemen yang digunakan untuk mendapatkan manfaat serta keuntangan terbaik. Pada penilitian yang dilakukan pada tahun 2014 berjudul *agile project management with scrum*: *case study of a Brazilian pharmaceutical company IT project* menyatakan bahwa dalam sudut pandang manajer projek ketika menggunakan pendekatan *agile* memberikan keuntungan dari segi biaya, *client*, dan projek itu sendiri. Dibanding dengan pendeketan tradisional, pendekatan *agile* dinilai memangkas 80% waktu pengerjaan projek dan 50% biaya yang dikeluarkan untuk kebutuhan projek jika dibandingkan dengan pendekatan tradisional. Hal tersebut dapat terealisasi dengan mengikuti alur kerja *scrum*, pembagian tim dalam pengembangan pada *scrum*, dan artifak pada metode *scrum* (Petter & Nils, 2014).

Dalam melakukan penugasan setiap individu memiliki keterampilan dan kecakapan terhadap bidangnya masing-masing sehingga pengambilan keputusan dalam pemilihan tugas harus dilakukan dengan tepat terhadap setiap anggota, tetapi terdapat beberapa pertimbangan yang dipikirkan untuk melakukan manajemen penugasan yaitu efisiensi pengerjaan, pembatasan waktu, pengontrolan pengerjaan, dan hasil yang optimal. Metode *scrum* dapat menjawab permasalahan dalam melakukan manajemen penugasan, metode *scrum* dipilih karena syarat dalam metode tersebut berbanding lurus dengan permasalahan dan kebutuhan manajemen penugasan. Pada penelitian sebelumnya terdapat beberapa kebutuhan yang harus dikembangkan untuk memenuhi kebutuhan dalam manajemen tugas, seperti perlunya seleksi terhadap individu untuk mengerjakan setiap *log*, komunikasi yang harus diterapkan untuk kelancaran pengerjaan projek, kontrol pada setiap aktivitas didalam projek, dan dokumentasi yang akan dikembangkan pada penilitian ini.

2. LANDASAN KEPUSTAKAAN

2.1. Rekayasa Perangkat Lunak

Pengertian dari rekayasa perangkat lunak yaitu ilmu yang dijadikan sebagai dasar untuk pengembangan, proses dan rekayasa pengembangan perangkat lunak berkaitan satu sama lain dari awal pengembangan sistem hingga pemeliharaan sistem, terdapat pendekatan sistematis rekayasa perangkat yang disebut proses perangkat lunak dengan 4 kegiatan mendasar yaitu *software development, software specification, software evolution, dan*

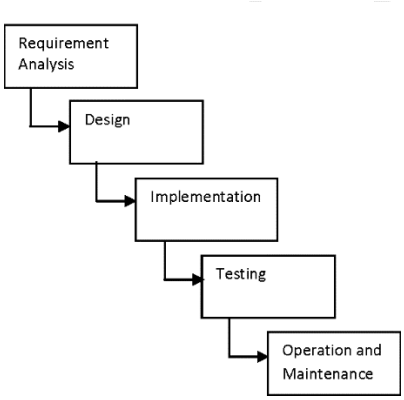
software validation (Sommerville, 2011).

2.2. Pengembangan Perangkat Lunak

Merupakan tahapan pemilihan model yang akan digunakan untuk mempermudah pengerjaan dalam perancangan perangkat lunak dengan Software Development Life Cycle (SDLC). SDLC secara konseptual merupakan kerangka kerja dalam pengembangan aplikasi, terdapat beberapa metode dari SDLC yang dapat diterapkan dalam perangkat lunak seperti waterfall, spiral, V-Model, dan B-Model (Ruparelia, 2010).

2.3. Model Pengembangan Perangkat Lunak

Pada setiap model memiliki siklus yang berbeda-beda dalam mencapai tujuan sehingga perlu pertimbangan dalam pemilihan model SDLC. Dalam penilitian ini model waterfall dipilih sebagai model atau dasar perancangan. Dengan menekankan pendekatan sistematis dimana pengembangan tidak boleh dilakukan ke tahap selanjutnya jika fase sebelumnya belum selesai. (Sommerville, 2011). Terdapat 5 fase di dalam waterfall model yang dapat dilihat dalam gambar 1.



Gambar 1 Model Waterfall

Sumber : (Adenowo & Adenowo, 2018)

2.4. Pendekatan Berorientasi Objek

Pemrograman berorientasi objek (OOP) adalah pendekatan yang memiliki fokus untuk melakukan cara agar objek dapat saling berinteraksi dalam berkomunikasi dan berbagi informasi. Paradigma OOP banyak digunakan dalam rekayasa perangkat lunak, terdapat beberapa konsep dari OOP seperti *class* dan *object, inheritance*, *messages*, dan *polymorphism*. OOP memberikan 3 tujuan diantaranya *extensibility*, *flexibility*, dan *reusability* dalam pengembangannya. *Reusability* berguna untuk *script* yang sudah dipakai untuk digunakan kembali. *Extensibility* merupakan sifat atau perilaku dari kelas dasar akan diturunkan ke kelas turunnya. *Flexibility* yaitu dapat menambahkan lebih banyak variasi dan modifikasi dengan menggunakan nama yang sama namun variabel yang berbeda (Urdhwareshe, 2016).

2.5. Pemodelan Berorientasi Objek

Unified Modeling Language (UML) merupakan pemodelan bahasa secara visual yang umum, semantik, dan kaya akan sintaks untuk merancang arsitektur sistem perangkat lunak yang kompleks baik secara struktural maupun perilaku. Dalam perancangannya UML memiliki hubungan langsung antara desain OOP dan analisis(*Lucid Chart*).

1. Usecase Diagram

Usecase diagram merupakan dasar dari sebuah bahasa pemodelan terpadu, sederhananya *usecase* memiliki peran untuk mengidentifikasi pelaku di dalamnya yang berinteraksi dan menjelaskan jenis interaksinya. Untuk saat ini *Unified Modeling Language (UML)* salah satu parameter yang relevan untuk membuat pemodelan berorientasi objek dengan tujuan penggunaan mendapatkan elisitasi persyaratan (Sommerville, 2011).

2. Usecase Scenario

Usecase scenario merupakan pemahanan tentang bagaimana seseorang dapat berinteraksi dengan sistem perangkat lunak, *usecase scenario* digunakan sebagai prasyarat untuk merumuskan sistem. Setiap skenario biasanya mencakup satu atau sejumlah kecil kemungkinan interaksi (Sommerville, 2011).

3. Sequence Diagram

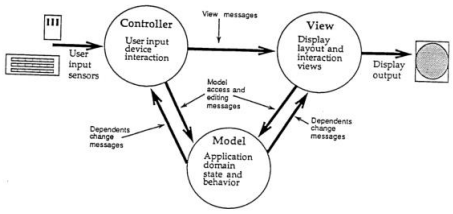
Sequence diagram digunakan dengan tujuan pemodelan intraksi antar aktor dan objek disebuah sistem. *Sequence diagram* menunjukan interaksi secara *sequential* selama *usecase* atau *usecase instance* tertentu. Objek dan aktor dalam suatu *sequence* biasanya tercantum ditujukan dengan panah yang bernotasi, sedangkan panah anotasi menunjukkan panggilan yang dilakukan ke objek, parameter, dan pengembalian nilai (Sommerville, 2011).

4. Class Diagram

Class diagram merupakan pengembangan model untuk menunjukan interaksi antar kelas dan objek dalam sebuah sistem. Di dalam *class diagram* memiliki asosiasi yang menghubungkan antar kelas di dalamnya yang menunjukan terdapat hubungan antar kelas satu dan yang lainnya. Diagram merupakan salah satu bagian dari UML yang memiliki tingkat detail yang berbeda dengan diagram lainnya (Sommerville, 2011).

2.6. Pola Perancangan Perangkat Lunak

Model-View-Controller (MVC) merupakan paradigma untuk mengimplementasikan komponen aplikasi yang interaktif. MVC terdiri dari 3 *superclass* yakni *view*, *model*, dan *controller* termasuk *subclass* yang ada didalamnya. Struktur MVC yang dikembangkan dalam pengembangan perangkat lunak bersifat *reuseablity* dan *pluggability* yaitu *class* yang dapat digunakan kemabli dan mendefinisikan *class* baru., untuk memahami konsep MVC dapat dilihat pada gambar 2 (Krasner & Pope, 1988).



Gambar 2 Kondisi Dan Alur Pesan MVC

Sumber : (Krasner & Pope, 1988)

2.7. Pengujian Perangkat Lunak

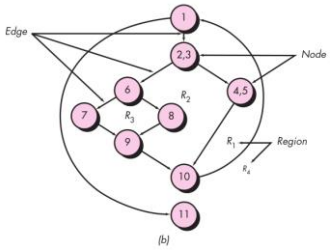
Pengujian perangkat lunak serangkaian metode untuk menguji seluruh perancangan dan aktivitas sistem yang dilakukan oleh tim pengembang. Pengujian dibantu oleh *tester* karena pengujian tidak harus dinilai dari satu sudut pandang sehingga memunculkan permasalah yang sebelumnya tidak diketahui. (Sommerville, 2011). Dilakukan 3 pengujian untuk menguji suatu perangkat diantaranya pengujian validas, unit, dan *usability*.

1. Pengujian Unit

Pengujian unit adalah pengujian komponen seperti metode, objek dan kelas. Untuk melakukan pengujian unit diperlukan beberapa contoh fungsi atau metode individual karena fungsi atau metode individual yang ada pada sistem merupakan jenis komponen yang paling sederhana (Sommerville, 2011). Fokus dari pengujian unit adalah verifikasi unit terkecil dari komponen desain perangkat lunak, dengan menguji struktur data dan logika pemrosesan internal dengan kompenen yang suadh ada. Metode yang digunakan adalah *white-box testing* (Pressman, 2010).

a. Flow Graph Anotation

Dalam pembuatan *flow graph*, setiap lingkaran dalam *flow graph* disebut *node* yang merepresentasikan lebih dari 1 pernyataan prosedural. Garis panah yang menghubungkan antar *graph* disebut sebagai *edge* untuk menggambarkan *flow*. Dan terdapat *area* dibatasi *edge* dan *node* disebut *region*. Terakhir adalah *predicted node* merupakan percabangan yang terjadi pada suatu *node*. Terkait detail dari pembuatan *flow graph* dapat dilihat pada gambar 3 (Pressman, 2010).



Gambar 3 Contoh Aliran Flowgraph

Sumber : (Pressman, 2010)

b. Independent Path

Jalur *independent* merupakan seluruh jalur yang memperkenalkan setidaknya 1 jalur baru untuk penyataan pemrosesan atau kondisi baru. Ketika dirancang dalam bentuk grafik suatu *independent path* harus melalui jalur yang belum dilalui sebelum di definisikan.

c. Cyclomatic Complexity

Merupakan metrik perangkat lunak yang memberikan ukuran logis dalam suatu program. Ketika digunakan dalam pengujian jalur dasar, nilai yang dihitung dalam *cyclomatic complexity* yaitu menentukan jumlah *independent path* dalam set dasar program dan memberikan batas bahwa seluruh jalur telah dites setidaknya satu kali. Perhitungan dapat dilakukan dari 1 dari 3 cara yang ada, yaitu :

i. Menghitung seluruh region yang ada didalam *flowgraph.*

ii. *Cyclomatic Complexity* V(G) di definisikan dengan E merupakan jumlah *edge* dari *flow graph* dan N merupakan jumlah *node*.

$$V(G) = E - N + 2$$

iii. *Cyclomatic Complexity* V(G di definisikan dengan P merupakan *predicted node* yang terdapat pada *flow graph.*

$$V(G) = P + 1$$

2. Pengujian Validasi

Merupakan pengujian yang mengarahkan kinerja sistem untuk menentukan keadaan sistem berjalan sebagaimana mestinya dengan melakukan serangkaian kasus uji (Sommerville, 2011). Salah satu metode yang dapat diterapkan untuk melakukan pengujian validasi adalah *black box testing* memiliki fokus untuk memenuhi persyaratan fungsional perangkat lunak (Pressman, 2010).

3. Pengujian Usability

Pengujian *usability* bertujuan untuk mengevaluasi tingkatan seorang *end-user* dapat secara efektif berinteraksi dengan sistem, menerima umpan balik yang diberikan oleh sistem, dan penilaian terkait interaksi yang konsisten. Pengujian ini umumnya digunakan oleh *User Experience Developer* untuk meneliti proses seorang *user* (pengguna) selama berinteraksi dengan sistem (Pressman, 2010).

3. METODOLOGI

Langkah-langkah dalam melakukan penelitian ini dapat dilihat dalam gambar 4.



Gambar 4 Diagram Alir Metodologi Penlitian

3.1. Studi Literatur

Dalam penelitian ini studi literatur merupakan tahapan pertama dari beberapa tahapan yang akan dilakukan, studi literatur dilakukan dengan mengkaji beberapa sumber informasi, teori, prinsip untuk dimasukan ke dalam penilitan ini.

3.2. Rekayasa Kebutuhan

Tahapan ini diperlukan untuk mendapatkan kebutuhan dari sistem yang akan dirancang dengan melakukan analisis kebutuhan dapat mengidentifikasi gambaran sistem, fungsionalitas dan batasan-batasan di dalamnya.

3.3. Perancangan Sistem

Perancangan sistem bertujuan untuk merancang *design*/arsitektur dan kompenen sistem dengan *Unified Model Language* (UML). Hasil dari perancangan adalah perancangan arsitektur, perancangan komponen, perancangan data, dan perancangan antarmuka.

3.4. Implementasi Sistem

Untuk melakukan implementasi dibutuhkan beberapa bahasa pemrograman seperti HTML, CSS, JS, PHP, dan bahasa pemrograman tambahan lainnya. Penelitian ini menggunakan beberapa framework untuk mendukung penelitian karena dinilai dokumentasi dalam *framework* yang lengkap sehingga pembuatan sistem dapat lebih cepat.

3.5. Pengujian Sistem

Penilaian yang dapat dilakukan dalam pengujian sistem agar pengujian tepat sasaran, dengan menggunakan teknik *black* dan *white testing*. *White box testing* merupakan pengujian yang bertujuan untuk mengetahui sistem sudah berjalan sebagaimana mestinya. Sedangkan pengujian menggunakan teknik *black box testing* yang bertujuan untuk megetahui setiap fungsi dalam sistem berjalan dengan baik.

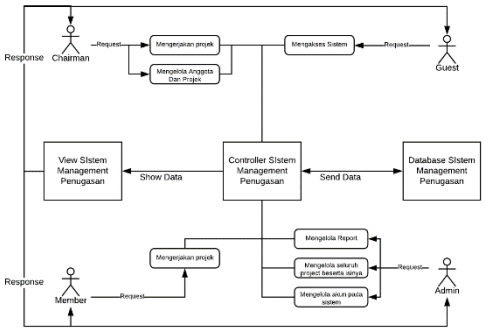
3.6. Kesimpulan Dan Saran

Pengambilan kesimpulan dan saran dilakukan terkahir kali saat semua prosedur dalam pembuatan sistem sudah dijalankan dengan baik. Kesimpulan dan saran akan mengeluarkan ouput berupa pernyataan bahwa sistem yang dibuat apakah sudah berjalan sesuai keinginan dan kebutuhan, bagian ini memiliki peran penting untuk pengembangan sistem kedepan agar dapat lebih baik.

4. REKAYASA KEBUTUHAN

4.1. Deskripsi Umum Sistem

Sistem ini berfokus untuk melakukan kontrol terhadap setiap progress dengan mengusung konsep *team-collaboration* dimana setiap anggota yang berada di dalam projek dapat melakukan *monitoring* terhadap pekerjaan anggota lain secara transparan sehingga progress dari masing-masing individu dapat dinilai. Gambaran umum sistem *management* penugasan berbasis web ditunjukkan pada gambar 5.



Gambar 5 Gambaran Umum Sistem

4.2. Identifikasi Aktor

Penjelasan mengenai aktor yang terdapat dalam sistem akan dijelaskan pada tabel 1.

Tabel 1Identifikasi Aktor

No	Aktor	Deskripsi
1	*Guest*	Merupakan aktor yang belum mempunyai otoritas serta data apapun di dalam sistem untuk melakukan akses di dalam *project*.
2	*Chairman*	Seorang *chairman* dapat mengundang *user* ke dalam suatu project dan menambakannya ke dalam *board* untuk dapat mengakses *activity* yang ada di dalam *board.*
3	*Member*	*Member* memiliki otoritas untuk memanipulasi *activity* di dalam suatu *board* dengan catatan user yang berkaitan telah masuk ke dalam suatu *board*.
4	*Admin*	Merupakan aktor yang mengatur segala bentuk kegiatan yang ada di dalam sistem. Pada umumnya dengan tujuan untuk mengatur berbagai kegiatan *project*, *user*, dan *report*.

4.3. Daftar Kebutuhan Fungsional

Setelah mendefinisikan setiap aktor dan deskripsinya selanjutnya pembuatan daftar kebutuhan yang dibedakan menjadi 2 bagian yaitu kebutuhan fungsionalitas dan spesifikasi kebutuhan. Sebanyak 47 fungsionalitas yang akan dibuat dalam sistem ini dengan spesifikasi kebutuhan terperinci untuk dibuat kedalam *usecase diagram*.

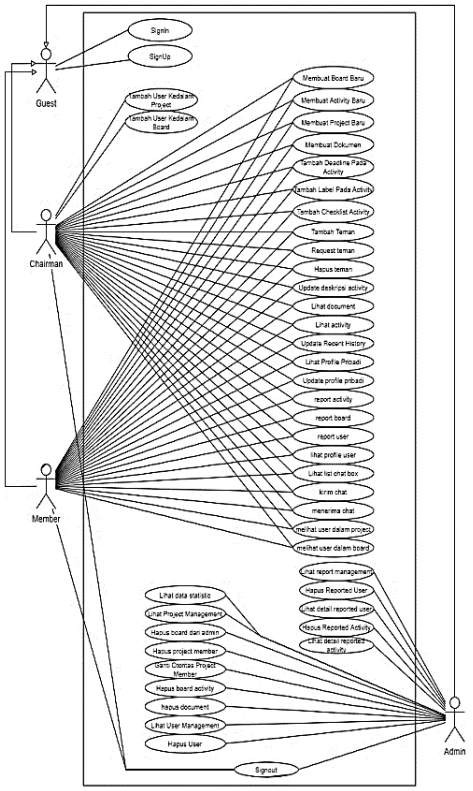
4.4. Daftar Kebutuhan Non-Fungsional

Kebutuhan non-fungsional digunakan untuk melakukan pengujian terhadap perancangan yang telah dilakukan, kebutuhan fungsional yang digunakan adalah *usability.*

4.5. Pemodelan Perangkat Lunak

1. Usecase Diagram

Usecase diagram berisikan aktor-aktor yang mampu mengakses ke dalam fungsional sistem, ada beberapa aktor di dalam sistem diantaranya guest, sistem, member. Untuk detail usecase diagram dapat dilihat dalam gambar 6.



Gambar 6 Usecase Diagram

2. Usecase Scenario

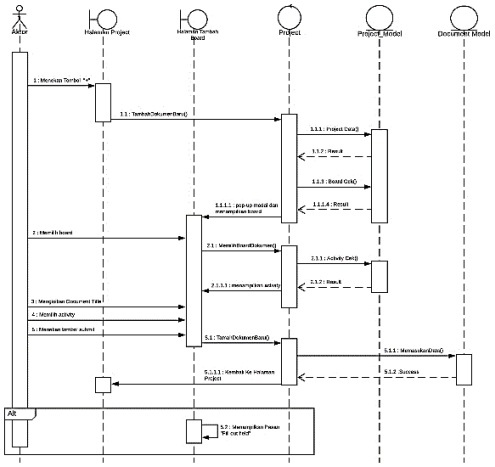
Use case scenario memiliki beberapa komponen diantaranya *objective*, aktor, *pre-condition*, *post-condition*, *main flow*, dan *alternative flow* dengan tujuan untuk menggambarkan alur dari suatu fungsional.

5. PERANCANGAN DAN IMPLEMENTASI

5.1. Perancangan Sistem

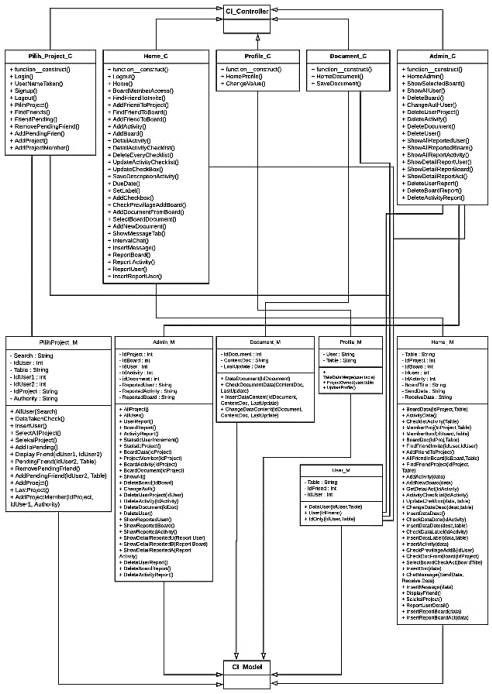
1. Perancangan Arsitektur

Tahap ini akan dijelaskan lebih rinci mengenai *sequence diagram* dan *class diagram* dari sistem *management* dalam melakukan penugasan berbasis *web*. *Sequence diagram* yang ditampilkan merupakan fungsi untuk menambahkan dokumen baru pada sistem, untuk gambar sequence terdapat pada gambar 7.



Gambar 7 Sequence Diagram Tambah Dokumen

Class diagram yang dirancang akan merepresentasikan hubungan antara setiap objek dengan *controller* dan *model* serta interaksi diantara setiap objek. Untuk gambar *class diagram* terdapat pada gambar 8.



Gambar 8 Class Diagram

2. Perancangan Kompenen

Perancangan komponen yang dibuat merupakan *pseudocode* yang mengambil contoh dari *method AddDocumentFromBoard()*. *Pseudocode* dipilih karena penggambaran kode tingkat tinggi untuk memudahkan implementasi kode. Untuk detail dari *method* terdapat pada tabel 2.

Tabel 2 *Pseudocode Add Document*

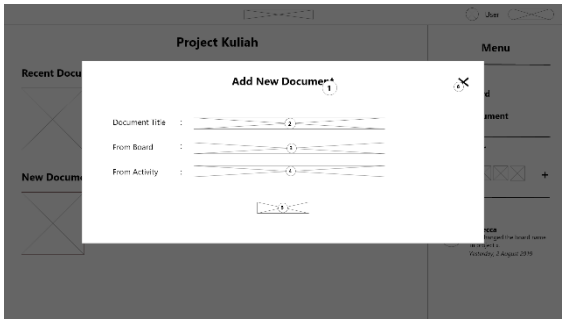
Pseudocode Method AddDocumentFromBoard
Mulai Inisialisasi variabel id_project = session_userdata(“Id_Project”) Inisialisasi variabel data = Pemanggilan *method CheckDocumentBasedOnBoard()* dari *board model* dengan membawa parameter variabel id_project Inisialisasi variabel output Foreach result variable data Inisialisasi variabel board_title = Menampilkan seluruh table yang dimiliki oleh project dalam select input dan membawa value End Foreach Inisialiasi variabel data_check = *memanggil method select_board_activity()* dari *board model* dengan membawa variabel (board_title) IF (data_check = null) Select option tidak menampilkan apapun Else Foreach data_check Menampilkan activity dari board yang dipilih untuk dipilih dalam select option End Foreach End If Echo variabel output untuk mengembalikan *value* ke view dari variabel *output* Selesai

3. Perancangan Data

Perancangan data dapat terbentuk dari setiap data yang telah dikumpulkan untuk membuatnya menjadi sebuah entitas yang memiliki relasi satu sama lain dalam bentuk *physical data model.*

4. Perancangan Antarmuka

Perancangan antarmuka merupakan *low-fidelity* dari gambaran tampilan antarmuka. Berikut gambaran tampilan antarmuka dari *AddDocumentFromBoard().*Tampilan dari perancangan antarmuka terdapat pada gambar 10.



Gambar 9 Tampilan Perancangan Antarmuka tambah dokumen

5.2. Implementasi Sistem

1. Implementasi Data

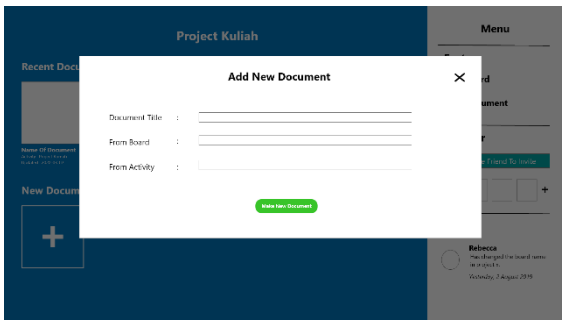
Implementasi data merupakan tahapan untuk membentuk penyimpanan data bagi sistem dalam rangka melakukan *data management.* Implementasi data dapat dibentuk dari perancangan data yang sudah dibuat pada rekayasa kebutuhan untuk di Implementasi. Implementasi data dibantu menggunakan *database* MySQL dan bahasa pemrograman SQL.

2. Implementasi Kode

Method AddDocumentFromBoard() pada *controller* bertujuan untuk menambahkan dokumen baru ke *project* dengan referensi *parent* dari suatu *activity*. Implementasi kode mengacu pada *pseudocode* pada perancangan komponen sebelumnya.

3. Implementasi Antarmuka

Implementasi antarmuka menambahkan dokumen berisikan input yang harus diisikan untuk diinput ke database. Untuk tampilan terdapat pada gambar 11.



Gambar 10 Implementasi Antarmuka *AddDocumentFromBoard()*

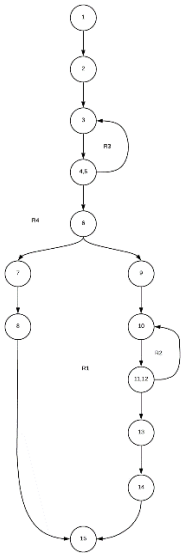
6. PENGUJIAN

6.1. Pengujian Unit

Pengujian ini berfokus menemukan jalur independen *flow graph* dan mendapat hasil *cyclomatic complexity*. pengujian ini menggunakan *method AddDocumentFromBoard()* dengan *basis path testing.*

1. *Flow Graph*

Pada gambar 12 dalam pengujian unit *AddDocumentFromBoard()* terdapat 13 *Nodes*, 15 *Edges*, *4 Region*, dan 3 *Predicted Nodes.*



Gambar 11 *Flow Graph Method AddDocumentFromBoard()*

2. *Cyclomatic Complexity*

V(G) = Jumlah region pada flow graph = 4

V(G) = (15 Edges – 13 Nodes) + 2 = 4

V(G) = 3 (Total Predited Nodes) + 1 = 4

3. Independent Path

Jalur 1 = 1-2-3-4-5-6-**7-8**-15

Jalur 2 = 1-2-3-4-5-6-**9-10-11-12-13-14**-15

6.1. Pengujian Validasi

Tabel 3 Pengujian Validasi Tambah Dokumen

Kode Kebutuhan	TD-F-10
Nama Kasus	Berhasil dapat menambah
Uji	*document*
Prosedur	1. Mengakses halaman *project* 2. Pada *side content* bagian fitur pilih dokumen 3. Menekan tombol "+" pada bagian bawah *content* 4. Sistem menampilkan *modal* untuk mengisikan *document title* dan memilih *board* pada *select option* 1. Aktor memilih *select option* pada *activity* 2. Aktor menekan tombol "*Add New Document"*
Expected Result	Sistem melakukan *re-direct* ke halaman *project* untuk melakukan *refresh*
Result	Sistem melakukan *re-direct* ke halaman *project* untuk melakukan *refresh*
Status	*Pass*

6.1. Pengujian *Usability*

Setelah pengisian kuisioner dilakukan dan data diperoleh, setelah itu melakukan penghitungan skor berdasarkan penilaian yang telah dilakukan oleh responden, perhitungan dilakukan dengan metode *System Usability Scale* (SUS). Dan terbagi menjadi 2 bagian yaitu responded yang sudah dan belum mengetahui sistem berbasis *team-collaboration.*

Tabel 4 Responden Belum Mengetahui *Team Collaboration*

No	Nama	Pertanyaan										Jumlah Skor	Hasil Akhir
		1	2	3	4	5	6	7	8	9	10		
1.	Anne Diane R	4	1	5	3	5	1	4	2	5	1	35	87.5
2.	Haidar Azmi	3	4	1	3	3	3	3	3	4	4	17	42.5
Rata-Rata Penilaian													65

Dan hasil dari responden yang sudah mengetahui *team-collaboration* adalah 74.2. Data tabel tidak dimasukin kedalam jurnal karena banyakanya jumlah data sebagai parameter penilaian.

7. KESIMPULAN DAN SARAN

7.1. Kesimpulan

Dari tahapan yang telah dilakukan dalam pengembangan sistem *management* dalam melakukan penugasan berbasis *web*, maka dapat ditarik kesimpulan bahwa:

1. Tahap dari analisis kebutuhan yang dilakukan pada sistem *management* dalam melakukan penugasan berbasis *web* menghasilkan 47 kebutuhan fungsional dimana setiap fungsional dirancang dalam bentuk *usecease diagram* dan *scenario* dan 1 kebutuhan non-fungsional dengan pendekatan *usability*.
2. Tahap dari perancangan yang dilakukan pada sistem *management* dalam melakukan penugasan berbasis *web* menghasilkan, perancangan komponen, perancangan arsitektur, perancangan antarmuka, dan perancangan data.
3. Tahap dari implementasi yang dilakukan pada sistem *management* dalam melakukan penugasan berbasis *web* menghasilkan implementasi data, implementasi kode, dan implementasi antarmuka.
4. Terakhir tahap pengujian yang dilakukan pada sistem *management* dalam melakukan penugasan berbasis *web* dilakukan mencakup pengujian unit, pengujian validasi, dan pengujian *usability*.

7.1. Kesimpulan

Untuk pengembangan sistem *management* dalam melakukan penugasan berbasis *web* agar menjadi sistem yang lebih baik, maka dapat diberikan beberapa saran, antara lain:

1. Pada halaman *admin*, fitur yang disediakan terlalu sedikit sehingga pengontrolan terhadap keseluruhan *project* masih minim. Dengan penambahan fitur diharapkan penggunaan sistem dapat lebih nyaman.
2. Penambahan *user guide* saat pertama kali *user* membuat *user* baru dan mengakses sistem sehingga potensi penggunaan seluruh fungsionalitas dapat dimaksimalkan.

8. DAFTAR PUSTAKA

Keshk, A. M., Maarouf, I., & Annany, Y. (2018). Special studies in management of construction project risks, risk concept, plan building, risk quantitative and qualitative analysis, risk response strategies. *Alexandria Engineering Journal*, *57*(4), 3179–3187.

Krasner, G. E., & Pope, S. T. (1988). A Description of the Model-View-Controller User Interface Paradigm in the Smalltalk-80 System. *Journal Of Object Oriented Programming*, *1*(3), 26–49.

Lei, H., Ganjeizadeh, F., Jayachandran, P. K., & Ozcan, P. (2017). A statistical analysis of the effects of Scrum and Kanban on software development projects. *Robotics and Computer-Integrated Manufacturing*, *43*, 59–67.

Petter, H., & Nils, K. (2014). International Journal of Managing Projects in Business Article information : *International Journal of Managing Projects in Business*, *7*(1), 133–143.

Pressman, R. S. (2010). Software-Engineering 7th ED by Roger S. Pressman. In *Software Engineering A Practitioner's Approach*.

Ruparelia, N. B. (2010). Software development lifecycle models. *ACM SIGSOFT Software Engineering Notes*, *35*(3), 8–13.

Sommerville. (2011). Software Engineering (Ii). In *Economic Computation and Economic Cybernetics Studies and Research* (Vol. 18).

Urdhwareshe, A. (2016). Object-Oriented Programming and its Concepts. *International Journal of Innovation and Scientific Research*, *26*(1), 1–6.

Nigele Ostime. (2017). 7 - Basic Management Techniques. *Handbook of Practice Management*, *106*(1), 1–48.